Competitive Programming

Section 44: String Algorithms II

Competitive Programming Course

2026-10-20

Table of contents

1 Three structures, three questions 3

2 Tries: prefixes of many words 4

3 Palindrome queries 5

4 Suffix automata: all substrings of one string 6

5 Implementation notes and tests 8

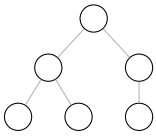
6 Exercises 9

1 Three structures, three questions

Section 43 had one object, the prefix function, and everything came out of it. This section is the opposite: three unrelated structures, and the skill being taught is choosing between them. Picking the wrong one is not a slow solution, it is usually an impossible one.

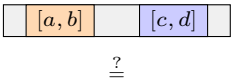
The question to ask first is never "which structure do I know?" but **what object is the query about?**

Trie
many strings,
shared prefixes



how many words start with ap?
walk $|p|$ edges

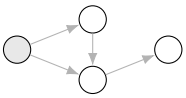
Rolling hash
one string,
compare two windows



$\stackrel{?}{=}$

are these two windows equal?
$O(1)$ after $O(n)$ prep,
probabilistic

Suffix automaton
one string,
every substring at once



how many distinct substrings?
$O(n)$ states
each groups several substrings
count by length (Chapter 4)

They are not interchangeable, and that is the whole skill here. A suffix automaton answers nothing useful about a *dictionary* of separate words — it is built from one string. A trie cannot compare two arbitrary substrings of one long text — it indexes prefixes, not positions. Before writing code, say out loud which object the query names: a prefix of a dictionary word, a window inside one text, or the set of all substrings. The answer picks the structure.

In the automaton panel, **one circle can represent several distinct substrings**. Counting circles therefore does not give the answer. Each state stores a consecutive range of substring lengths, with one substring per length. The formula introduced in Chapter 4 counts the lengths in each state, then adds these counts. For example, in `aba`, one state represents both `ba` and `aba`: that single state contributes two substrings.

The query names	Reach for	Section problem
a prefix of a dictionary word	trie	A, D
a window inside one text	rolling hash, or direct comparison	B
the set of all substrings of one text	suffix automaton or suffix array	C

Each of the three chapters below takes one row of that table.

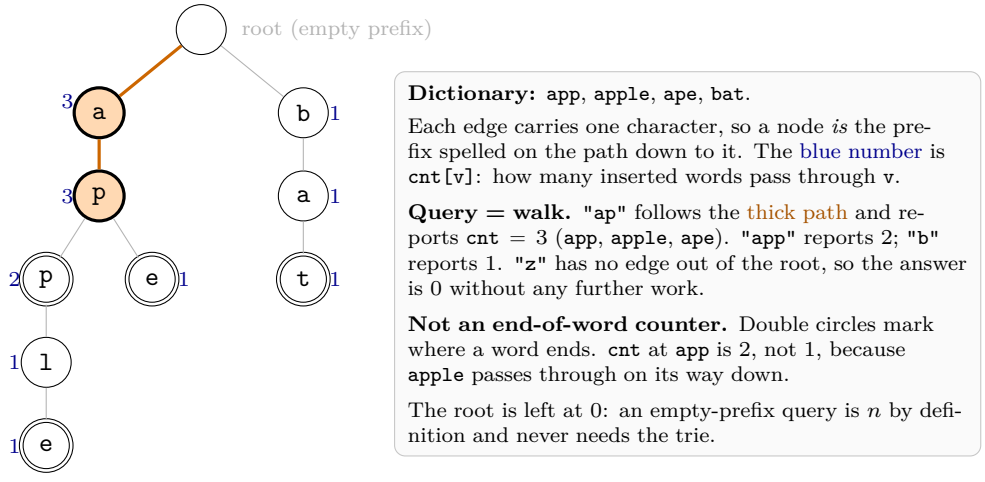
2 Tries: prefixes of many words

A trie is a rooted tree whose edges carry single characters, so a node *is* the prefix spelled along the path from the root to it. Shared prefixes are stored once, which is the entire point.

To answer "how many dictionary words begin with `p`", store on each node the number of inserted words whose path passes through it:

```
insert(word):
    node = root
    for c in word:
        create edge c if missing
        node = child[node][c]
        cnt[node] += 1

query(p):
    node = root
    for c in p:
        if edge c missing: return 0
        node = child[node][c]
    return cnt[node]
```



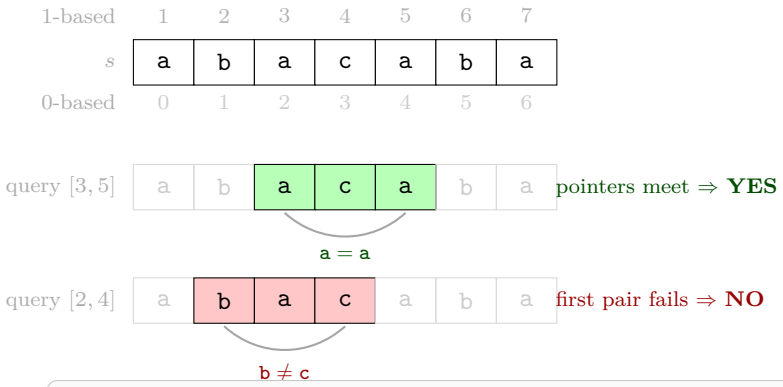
Two details decide whether this is correct.

`cnt` is not an end-of-word counter. It counts words *passing through*, so a node can hold a count larger than the number of words ending there. In the figure, `app` has `cnt = 2` because `apple` continues below it. If you need to know how many words *end* at a node, that is a second, separate counter.

The root is never incremented. The loop moves to a child before incrementing, so `cnt[root]` stays `0`. The empty prefix is shared by all `n` words and no query needs the trie to discover it. A missing edge ends a query immediately with `0`, with no need to walk the rest of the pattern.

3 Palindrome queries

For a substring `s[l..r]`, the direct test walks two pointers inward and stops at the first disagreement:



Convert the indices first. Statements here give $[l, r]$ 1-based and inclusive; arrays are 0-based. Subtract one from each end *before* comparing, and compare $s[l]$ with $s[r]$, then move inward while $l < r$. A single off-by-one turns every odd-length answer wrong while leaving even-length ones correct, which is why both parities must be tested.

Direct or hashed. The direct scan is $O(r - l + 1)$ per query and exact, which is why the local bounds allow it. For large query loads compare a forward hash of $[l, r]$ against a reverse hash of the mirrored range $[n-1-r,\ n-1-l]$ — fast, but probabilistic; Manacher or a palindromic tree is the deterministic alternative.

```
while l < r:
    if s[l] != s[r]: return NO
    l += 1
    r -= 1
return YES
```

This is `O(r-l+1)` per query and exactly right, which is why the local bounds allow it. It is also completely transparent: the equality condition is visible rather than hidden behind a hash.

When the query load is too large for that, compare a forward hash of `[l,r]` against a reverse hash of the mirrored range:

```
hash_forward(l, r) == hash_reverse(n-1-r, n-1-l)
```

Hashing is `O(1)` per query after `O(n)` preprocessing, but it is probabilistic. Manacher's algorithm and palindromic trees are the deterministic alternatives. When the constraints permit the direct check, prefer it.

4 Suffix automata: all substrings of one string

A suffix automaton is a compressed automaton that accepts exactly the substrings of one string, in `O(n)` states. Two arrays describe it:

- `len[v]`, the length of the longest substring represented by state `v`;
- `link[v]`, the state representing the longest proper suffix that lives in a different state.

A state groups substrings that have exactly the same **ending positions** in the original string. These are positions where an occurrence finishes, not where it starts. For `aba`, using 1-based positions:

- `a` ends at positions 1 and 3, so it has its own state.
- `b` and `ab` both end only at position 2, so they share a state.
- `ba` and `aba` both end only at position 3, so they share a state.

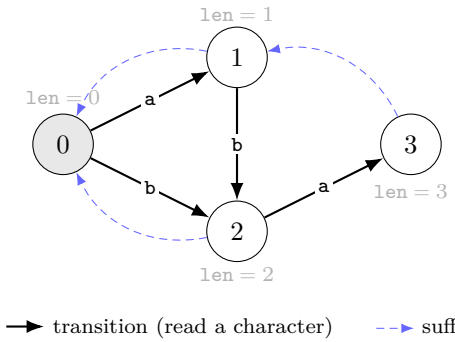
Within one state, there is exactly one represented substring for each length from `len[link[v]] + 1` through `len[v]`, inclusive. Why at most one per length? Choose any common ending position: the length and that ending position uniquely identify the substring. These lengths are consecutive because removing leading characters preserves the ending-position set until we cross into the suffix-link state.

Thus, if `a = len[link[v]]` and `b = len[v]`, the lengths belonging to `v` are `a + 1, a + 2, ..., b`. There are `b - a` of them. **The subtraction counts substrings represented by this state; it does not count their occurrences.**

For the state containing `ba` and `aba`, the longest string has length 3. Its suffix link goes to the state for `a`, whose maximum length is 1. So its contribution is `3 - 1 = 2`: lengths 2 and 3, namely `ba` and `aba`. The shorter suffix `a` belongs to another state and is counted there.

Every distinct nonempty substring belongs to exactly one state. Add the contributions of all states except the root:

```
answer = sum over v != root of ( len[v] - len[link[v]] )
```



Suffix automaton of `aba`

v	`len`	`link`	`len`$[v]$ − `len`[`link`]	represents
1	1	0	1	`a`
2	2	0	2	`b`, `ab`
3	3	1	2	`ba`, `aba`
		total	**5**	

Enumerating by hand gives the same five: `a`, `b`, `ab`, `ba`, `aba`.

One state holds a whole run of lengths that end at the same set of positions, and `link` points at the next shorter run. So the state adds exactly the lengths its link does not already cover — no substring is counted twice.

State 0 is excluded: it is the empty string.

The reason this does not double count is worth stating plainly: a single state holds a whole *run* of lengths whose occurrences end at the same set of positions, and `link[v]` points at the state holding the next shorter run. So state `v` contributes exactly the lengths its link does not already cover. The root is excluded because it represents the empty string.

Check the formula against a string small enough to enumerate. `ababa` has distinct substrings `a, b, ab, ba, aba, bab, abab, baba, ababa`, nine of them, and the automaton's contributions sum to nine. `aaa` gives `a, aa, aaa`, three.

Cloning. When an existing transition would break the invariant `len[p] + 1 == len[q]`, construction splits `q` into a clone carrying the same transitions but a shorter `len`, then redirects the affected suffix links. This is not an implementation trick; it is what keeps one state from representing two different sets of end positions. `abb` is the shortest string that forces one.

At the bounds used locally, problem C can also be solved by a set of substrings or a suffix array. The formula above is still the thing to know, because it is what scales.

5 Implementation notes and tests

Tries in C++. For lowercase input, `vector<array<int,26>>` with node `0` as the root and `0` meaning "absent" is compact and fast. Reserve capacity for the total input length; that is the exact upper bound on node count.

Tries in Python. A `dict` per node is clearer and usually faster than a 26-slot list for sparse alphabets. Build children with `setdefault`.

Everything else. Read input with `sys.stdin.buffer.read()`. For hashing, use two independent moduli or a random base — a single fixed 32-bit modulus is attackable. Palindrome endpoints arrive 1-based and inclusive; convert once, at the point of reading, not repeatedly inside the loop.

Case	What it catches
a word that is a prefix of another word (`app`, `apple`)	treating `cnt` as an end-of-word counter
duplicate words in the dictionary	a unique-prefix answer that must be `-1`
query prefix absent from the trie	a missing-edge path that walks past the end
single-character palindrome query	the `l < r` loop bound
even-length palindrome (`abba`)	an off-by-one that only breaks one parity
odd-length palindrome (`aba`)	the same off-by-one, the other parity
all-equal string (`aaaa`)	the maximal-state-sharing case
one-character string	an empty contribution sum

Useful values to check by hand: the trie over `app, apple, ape, bat` gives `cnt["ap"] = 3` and `cnt["app"] = 2`; `abacaba` answers NO for `[2,4]` and YES for `[3,5]`; and `aba` has exactly `5` distinct non-empty substrings.

Run the local reference suite with:

```
CP_TARGET=solution python3 sections/44_string_algorithms_ii/check.py
```

6 Exercises

Use A to fix the trie invariant before writing anything: `cnt[v]` counts words passing through `v`, not words ending at `v`. Use B to convert the 1-based inclusive endpoints exactly once and to test both parities. Use C to state the `len[v] - len[link[v]]` invariant even if you solve it with a set at these bounds, because that sentence is what transfers to large inputs. Use D as the blind synthesis problem: the shortest unique prefix of a word is the first node on its path whose count has fallen to one, so decide what the trie must store and what it means when the count never falls. Test a word that is a prefix of another, duplicate words, a prefix that is absent, and single-character inputs.

- A. Trie Prefix Count: Given dictionary words and prefix queries, count words having each prefix.
- B. Palindrome Queries: For each substring query, print whether the substring is a palindrome.
- C. Distinct Substrings: Print the number of distinct non-empty substrings of a string.
- D. Unique Prefix Lengths: For each word, print the length of its shortest prefix shared by exactly one input word, or `-1`.